

singly_linked_list.c

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node* next;
};

struct Node* head = NULL;

struct Node* createNode(int value) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = value;
    newNode->next = NULL;
    return newNode;
}

void insertAtBeginning(int value) {
    struct Node* newNode = createNode(value);
    newNode->next = head;
    head = newNode;
    printf("%d inserted at beginning\n", value);
}

void insertAtEnd(int value) {
    struct Node* newNode = createNode(value);
    if (head == NULL) head = newNode;
    else {
        struct Node* temp = head;
        while (temp->next != NULL) temp = temp->next;
        temp->next = newNode;
    }
    printf("%d inserted at end\n", value);
}

void insertAtPosition(int value, int pos) {
    if (pos == 1) { insertAtBeginning(value); return; }
    struct Node* newNode = createNode(value);
    struct Node* temp = head;
    for (int i = 1; i < pos - 1 && temp != NULL; i++) temp = temp->next;
    if (temp == NULL) { printf("Invalid position\n"); free(newNode); }
    else {
        newNode->next = temp->next;
        temp->next = newNode;
        printf("%d inserted at position %d\n", value, pos);
    }
}

void deleteAtBeginning() {
    if (head == NULL) { printf("List is empty\n"); return; }
    struct Node* temp = head;
```

```

        head = head->next;
        printf("Deleted %d from beginning
", temp->data);
        free(temp);
    }

void deleteAtEnd() {
    if (head == NULL) { printf("List is empty
"); return; }
    if (head->next == NULL) {
        printf("Deleted %d from end
", head->data);
        free(head); head = NULL; return;
    }
    struct Node* temp = head;
    while (temp->next->next != NULL) temp = temp->next;
    printf("Deleted %d from end
", temp->next->data);
    free(temp->next);
    temp->next = NULL;
}

void deleteByValue(int value) {
    if (head == NULL) { printf("List is empty
"); return; }
    if (head->data == value) { deleteAtBeginning(); return; }
    struct Node* temp = head;
    while (temp->next != NULL && temp->next->data != value) temp = temp->next;
    if (temp->next == NULL) printf("%d not found
", value);
    else {
        struct Node* toDelete = temp->next;
        temp->next = temp->next->next;
        printf("Deleted %d
", toDelete->data);
        free(toDelete);
    }
}

void display() {
    if (head == NULL) { printf("List is empty
"); return; }
    struct Node* temp = head;
    printf("List: HEAD -> ");
    while (temp != NULL) { printf("%d -> ", temp->data); temp = temp->next; }
    printf("NULL
");
}

void search(int value) {
    struct Node* temp = head; int pos = 1;
    while (temp != NULL) {
        if (temp->data == value) { printf("%d found at position %d
", value, pos); return; }
        temp = temp->next; pos++;
    }
    printf("%d not found
", value);
}

```

```

int main() {
    int choice, value, pos;
    while (1) {
        printf("
--- Singly Linked List Menu ---
");
        printf("1. Insert at Beginning
2. Insert at End
3. Insert at Position
");
        printf("4. Delete at Beginning
5. Delete at End
6. Delete by Value
");
        printf("7. Display
8. Search
9. Exit
Enter choice: ");
        scanf("%d", &choice);
        switch (choice) {
            case 1: printf("Enter value: "); scanf("%d", &value); insertAtBeginning(value);
break;

            case 2: printf("Enter value: "); scanf("%d", &value); insertAtEnd(value); break;
            case 3: printf("Enter value and position: "); scanf("%d %d", &value, &pos);
insertAtPosition(value, pos); break;
            case 4: deleteAtBeginning(); break;
            case 5: deleteAtEnd(); break;
            case 6: printf("Enter value to delete: "); scanf("%d", &value); deleteByValue(value);
break;

            case 7: display(); break;
            case 8: printf("Enter value to search: "); scanf("%d", &value); search(value);
break;

            case 9: exit(0);
            default: printf("Invalid choice
");
        }
    }
    return 0;
}

```